

# On the Effectiveness of Data Balancing Techniques in the Context of ML-Based Test Case Prioritization

Jediael Mendoza  
Carleton University  
Ottawa, ON, Canada  
jediamendoza@cmail.carleton.ca

Jason Mycroft  
Carleton University  
Ottawa, ON, Canada  
jasonmycroft@cmail.carleton.ca

Lyam Milbury  
Carleton University  
Ottawa, ON, Canada  
lyammilbury@cmail.carleton.ca

Nafiseh Kahani  
Carleton University  
Ottawa, ON, Canada  
nafiseh.kahani@carleton.ca

Jason Jaskolka  
Carleton University  
Ottawa, ON, Canada  
jason.jaskolka@carleton.ca

## ABSTRACT

Regression testing is the cornerstone of quality assurance of software systems. However, executing regression test cases can impose significant computational and operational costs. In this context, Machine Learning-based Test Case Prioritization (ML-based TCP) techniques rank the execution of regression tests based on their probability of failures and execution time so that the faults can be detected as early as possible during the regression testing. Despite the recent progress of ML-based TCP, even the best reported ML-based TCP techniques can reach 90% or higher effectiveness in terms of Cost-cognizant Average Percentage of Faults Detected (APFD<sub>C</sub>) only in 20% of studied subjects. We argue that the imbalanced nature of used training datasets caused by the low failure rate of regression tests is one of the main reasons for this shortcoming. This work conducts an empirical study on applying 19 state-of-the-art data balancing techniques for dealing with imbalanced data sets in the TCP context, based on the most comprehensive publicly available datasets. The results demonstrate that data balancing techniques can improve the effectiveness of the best-known ML-based TCP technique for most subjects, with an average of 0.06 in terms of APFD<sub>C</sub>.

## CCS CONCEPTS

• Software and its engineering → Empirical software validation.

## KEYWORDS

Test Case Prioritization, TCP, Regression Testing, ML-based TCP, Data Balancing

### ACM Reference Format:

Jediael Mendoza, Jason Mycroft, Lyam Milbury, Nafiseh Kahani, and Jason Jaskolka. 2022. On the Effectiveness of Data Balancing Techniques in the Context of ML-Based Test Case Prioritization. In *Proceedings of the 18th*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

PROMISE '22, November 17, 2022, Singapore, Singapore

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9860-2/22/11...\$15.00

<https://doi.org/10.1145/3558489.3559073>

*International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '22), November 17, 2022, Singapore, Singapore. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3558489.3559073>*

## 1 INTRODUCTION

Continuous Integration (CI) practices enable software developers to integrate their work as often as possible with the mainline codebase. Each integration is built and validated by automated regression testing [18]. Software companies widely practice CI due to the benefits it offers, notably, timely feedback to developers, early detection of faults, and fewer integration problems [26].

Regression testing is the cornerstone of quality assurance for software systems in the CI context [16, 25]. However, executing regression test cases can impose significant computational and operational costs. As reported, the execution of test cases, specifically for large software systems, requires significant computing resources [16, 25]. For example, a project at Microsoft has nearly 65,000 regression test cases, which take approximately three days to be fully executed [2].

Test Case Prioritization (TCP) techniques deal with the long execution time of regression test cases by prioritizing (ranking) the execution of test cases according to the optimal ranking that usually gives a higher execution priority to test cases with a higher probability of fault detection and lower execution times [41]. Depending on the execution budget, top-ranked test cases can be selected for the execution, or regression testing stops when a test case fails. Recently many researchers have applied Machine Learning (ML) in the TCP context (ML-based TCP) to rank test cases as close as possible to the optimal ranking.

Despite the recent progress of ML-based TCP techniques [5, 54], the best ML-based TCP techniques could not achieve high accuracy for all systems. In the most recent and comprehensive study by Yaraghi et al. [54], the best ML-based TCP techniques (pairwise ranking with Random Forest [10]) can reach 90% or higher effectiveness in terms of Cost-cognizant Average Percentile Fault Detected (APFD<sub>C</sub> [44]) only in 5 out of 25 studied projects. We argue that the imbalanced nature of training datasets used for TCP is one of the main reasons for this shortcoming. More specifically, ML-based TCP techniques train ML models based on datasets collected from the previous execution of regression test cases, the verdict (the result of the execution that can be passed or failed) of which is the main variable to predict. However, the regression test cases

rarely fail [3, 8, 32]. According to a comprehensive analysis of 1, 108 Java projects with regression testing [8], the ratio of builds with at least one failed test case to all builds has a median of 2.9%. The low failure rate of regression test cases leads to imbalanced datasets for the training of ML-based TCP techniques that negatively affect the accuracy of the trained ML models [24].

In general, there are two groups of techniques to deal with imbalanced datasets in the ML context. First are *algorithm-level techniques* that modify existing ML algorithms to alleviate their bias towards majority class [24] (e.g., weighted decision trees). Second are *data balancing techniques* that use sampling techniques (e.g., Random Over-sampling [24]) to balance the training datasets, and make them suitable for the normal ML techniques. In this work, we explore the application of known data balancing techniques for dealing with imbalanced data sets in the TCP context.

The data balancing techniques are classified into three groups: *over-sampling* techniques that increase the number of samples for the minority class, *under-sampling* techniques that decrease the of samples for the majority class, and *hybrid* techniques that perform both under-sampling and over-sampling techniques. In this work, we have reviewed related data balancing techniques, and selected 19 of state-of-the-art techniques that are implemented by the *Imbalanced-learn* toolbox [34]. We then conducted extensive experiments using the best ML-based TCP technique, and the most comprehensive publicly available datasets to answer the following research questions:

**RQ1.** How do data balancing techniques perform in the TCP context?

**RQ2.** How does the application of data balancing techniques impact the effectiveness of the ML-based TCP techniques in terms of APFDC?

Results show (1) over-sampling techniques perform better than under-sampling techniques, (2) the application of data balancing techniques improves the effectiveness of the best-known ML-based TCP technique for most (23 out of 25) subjects, ranging from 0.01 to 0.40 in terms of APFDC, (3) there is no single technique that provides the best results across all subjects, which means they need to be evaluated before being applied for a particular subject.

The rest of this paper is organized as follows. Section 2 briefly discusses ML-based TCP in the CI context, as well as data balancing techniques for dealing with imbalanced datasets. We review related work in Section 3. We then, in Section 4, describe our research questions, the evaluation approach (datasets and experiments) to answer each question, followed by the results. We finally conclude the paper in Section 5.

## 2 BACKGROUND

### 2.1 Test Case Prioritization

Test Case Prioritization (TCP) techniques aim to find an ordering for the test case executions close to the optimal ordering to detect faults as early as possible. Thus, executing a small fraction of a prioritized test suite may reduce the cost and time of testing, while detecting most of the faults. In this work, our focus is ML-based TCP techniques in the context of Continuous Integration (CI) of a software system where at each CI cycle, the system is built and released upon successful testing, including regression testing.

**Definition 1. (CI and CI Cycles).** We assume the CI history of a software system is captured as a sequence of cycles  $c_i$ ,  $1 < i < n$ ,

where  $c_1$  and  $c_n$  refer to the first and current cycles, respectively. A cycle  $c$  is a tuple  $\langle T, f \rangle$ , where  $T$  is a set of test cases, and  $f$  is a logical value that indicates whether or not the cycle has failed. The number of test cases in different cycles varies. A cycle can fail due to several reasons, including compilation errors or a test case failure. However, in this work, we are only interested in the latter and failed cycles are, in our experimental datasets, cycles with at least one failed test case.

**Definition 2. (Test Case Features).** To allow the application of ML, each test needs to be captured using a set of features. The features include, but are not limited to, the execution history of a test case, test case coverage, test case code complexity, etc. In this work, we rely on a comprehensive set of features (150 features) that is defined and collected by Yaraghi et al. [54], for several software systems in the CI context. The feature set captures many aspects of tests, including execution history of tests, source code of tests, coverage of tests, source code of the System Under Test (SUT), and process and complexity metrics related to the source code of tests and the SUT. The full description of features can be found in [54].

**Definition 3. (Optimal Ranking).** We define the optimal order ( $s_o$ ) of a set of test cases  $T$  as a sequence in which:

$$\begin{aligned} \forall t_1, t_2 \in T, \\ rank(s_o, t_1) < rank(s_o, t_2) \iff t_1.v > t_2.v \text{ or} \\ (t_1.v = t_2.v \text{ and } t_1.e \leq t_2.e), \end{aligned}$$

where function  $rank(s, t)$  returns the rank of a test case  $t$  in sequence  $s$ .

The above condition implies that, in the optimal ranking of the test cases, (1) all failing test cases (their verdict is one) are executed before passing test cases, (2) test cases with lower execution time are executed earlier than other test cases with the same verdict.

**Definition 4. (Test Case Prioritization (TCP)).** Regardless of underlying techniques for test case prioritization, we assume a TCP technique to be a ranking function that takes as an input a set of test cases' features, and returns an ordered sequence in which the position of the test cases shows their priority for execution, i.e., the test case with the lowest index (rank) gets executed first.

### 2.2 Data Balancing Techniques

ML techniques generally assume that all classes have the same number of samples, i.e., the distribution of the data is balanced. However, in many contexts, such as TCP, the distribution of data is skewed since samples corresponding to some of the classes are more frequent [24]. Consequently, most ML techniques are biased toward the majority class. In this context, data balancing techniques involve preprocessing data to make it more suitable for ML techniques. Data balancing techniques are simple and easy to configure, and they can be used in conjunction with any ML technique. Generally, data balancing techniques can be divided into three groups as discussed below.

**Under-sampling** techniques reduce the number of samples in the majority class. They offer either *fixed under-sampling*, or *cleaning under-sampling*. In the former, under-sampling is carried out to obtain an appropriate balancing ratio between the majority and the minority class. Various methods can be used to accomplish this, including random selection, clustering, and nearest neighbours rule

**Table 1: Data balancing techniques for dealing with imbalanced datasets.**

| Group          | Technique                                            |
|----------------|------------------------------------------------------|
| Under-sampling | <i>Random Under-sampling (RU)</i>                    |
|                | <i>Cluster Centroids (CC) [55]</i>                   |
|                | <i>Near Miss (NM) [36]</i>                           |
|                | <i>Edited Nearest Neighbour (ENN) [52]</i>           |
|                | <i>Repeated Edited Nearest Neighbours (REN) [49]</i> |
|                | <i>All KNN (AKN) [49]</i>                            |
|                | <i>Condensed Nearest Neighbour (CNN) [22]</i>        |
|                | <i>One Sided Selection (OSS) [31]</i>                |
|                | <i>Neighbourhood Cleaning Rule (NCR) [33]</i>        |
| Over-sampling  | <i>TomekLinks (TL) [50]</i>                          |
|                | <i>Instance Hardness Threshold (IHT) [46]</i>        |
|                | <i>Random Over-sampling (RO)</i>                     |
|                | <i>SMOTE (SM) [12]</i>                               |
|                | <i>SVM SMOTE (SSM) [39]</i>                          |
|                | <i>KMeans SMOTES (KSM) [12]</i>                      |
|                | <i>Borderline SMOTE (BSM) [21]</i>                   |
| Hybrid         | <i>ADASYN [23]</i>                                   |
|                | <i>SMOTE EEN (SME) [7]</i>                           |
|                | <i>SMOTE Tomek (SMT) [6]</i>                         |

(e.g., NearMiss [36]). Fixed under-sampling can result in removing necessary samples. Thus, cleaning under-sampling methods do not aim for the balancing ratio precisely, but rather attempt to clean the feature space according to some empirical criteria often derived from the nearest neighbours rule, namely: Condensed Nearest Neighbours (CNN) [22], Edited Nearest Neighbours (ENN) [52], One-Sided Selection (OSS) [31], and Neighbourhood Cleaning Rule (NCR) [33].

**Over-sampling** techniques add new samples to the minority samples to reach the specific ratio. To reach the ratio, they either over-sample by randomly replicating the samples from the minority class, or use the Synthetic Minority Over-sampling Technique (SMOTE) to create new artificial minority samples, based on the feature space similarities between existing minority samples [24]. The SMOTE has been studied in different variants, including SMOTE borderline [21] and SMOTE SVM [39]. Also, recently Generative Adversarial Techniques are used to generate samples. Shu et al. [45] has shown that using GANs can provide an improvement over SMOTE.

**Hybrid** data balancing techniques combine under-sampling with over-sampling to avoid the over-fitting that over-sampling can cause. For example, Batista et al. [6] combined SMOTE either with Tomek links or edited nearest neighbours (i.e., cleaning under-sampling methods) to prevent over-fitting.

Table 1 shows a list of data balancing techniques that are evaluated in this study. Techniques may perform differently depending on the characteristics of datasets. As a result, in this work, we do not rule out any existing techniques whose implementation is supported by the *Imbalanced-learn* toolbox [34].

### 3 RELATED WORK

Lima et al. [35] have presented a comprehensive review of existing TCP techniques that can be classified into two groups: heuristic and ML-based techniques.

Studies in the former apply heuristics to TCP that are defined primarily based on test coverage and execution history. Coverage-based heuristics are based on the principle that increasing structural test coverage of the SUT increases the chances of fault detection [30]. Extracting precise coverage information at a reasonable cost is the major drawback of coverage-based heuristics.

Heuristics that are based on test case execution history assume that previously failed test cases are more likely to fail again. For example, Kim and Porter [30] proposed an execution history heuristic that calculates ranking scores based on the average failure rate of test cases. The execution history is easily accessible, and some studies such as [17, 47], reported that the effectiveness of execution history heuristics is almost the same as ML-based TCP. However, the most recent study by Yaraghi et al. [54] shows that an ML techniques trained based on a comprehensive set of features performs significantly better than heuristics based on test case execution history.

ML-based TCP techniques train ML models based on features collected from different sources, such as test execution history, to prioritize test cases. Compared to heuristics that are often defined based on a single feature, ML models can be trained on a set of features from different sources. This enables ML models to deal with the complexities of the TCP problem for complex systems. Pan et al. [40] surveyed ML-based TCP techniques and reported that existing ML-based papers investigated a wide variety of ML techniques including reinforcement learning [9, 47], clustering [1], ranking models [9], and natural language processing [28]. Most existing work evaluated the TCP techniques based on subjects with very low numbers of failed test cases and short regression testing time. Having a sufficient number of failed builds is important for evaluating such TCP techniques, and creating the balanced datasets required by many ML techniques. Further, applying TCP techniques to systems with short regression testing time is not practically beneficial nor representative of real-world applications. A recent study by Yaraghi et al. [54] has addressed the above-mentioned issues partially by investigating the use of a comprehensive feature set (150 features) for training ML-based TCP models based on 25 realistic subjects. However, the trained ML models for TCP have not yet achieved satisfactory results in all cases. This paper complements the existing ML-based TCP techniques by improving the training datasets, and accounting for the imbalanced nature of training datasets. To the best of our knowledge no previous work studied the impact of data balancing techniques in the TCP context.

Furthermore, previous studies that employ data balancing techniques in software engineering contexts are closely related to ours [27, 43, 48]. For example, Tantithamthavorn et al. [48] investigate four commonly-used data balancing techniques in the context of defect prediction models. They conclude from an experimental analysis that quality assurance teams can use data balancing techniques to increase the completeness of the identification of software defects (i.e., Recall).

## 4 EMPIRICAL EVALUATION

In this section, we discuss the experiments for evaluating the effects of applying data balancing techniques on the effectiveness of ML-based TCP techniques. First, we outline and motivate two research questions. Then, we describe the Cost-cognizant Average Percentage of Faults Detected (APFD<sub>C</sub>) as the evaluation metric, and the subjects of the study. Finally, we explain the process and results of our experiments and discuss their practical implications.

### 4.1 Research Questions

**RQ1.** *How do the selected state-of-the-art data balancing techniques (Table 1) perform in the context of TCP?*

- **RQ1.1** Which combinations of data balancing techniques and balancing ratio perform best?
- **RQ1.2** Does the balancing ratio impact the effectiveness of the balancing techniques?
- **RQ1.3** Which group (under-sampling, over-sampling, hybrid) of the data balancing techniques perform best across all subjects?

In this RQ, we are interested in examining whether a specific configuration of data balancing technique and balancing ratio outperforms others across all subjects. Answering this question has practical value, since it provides evidence to recommend a specific data balancing technique in the context of TCP. Also, we examine whether or not the balancing ratio that is set for a group of sampling techniques impacts their effectiveness. Finally, as discussed in Section 2, there are three groups of data balancing techniques. Our goal is to examine whether a specific group performs better than others across all subjects in the TCP context, where for most subjects a minority class (failed test cases) has significantly fewer samples than the majority class (passed test cases).

**RQ2.** *How does the application of data balancing techniques impact the effectiveness of the ML-based TCP techniques in terms of APFD<sub>C</sub>?*

- **RQ2.1** How does the application of the best data balancing technique across all subjects impact the TCP effectiveness?
- **RQ2.2** How does the application of the best data balancing technique per each subject impact the TCP effectiveness?

This question complements RQ1 by determining if applying the best data balancing techniques selected based on the RQ1 results can increase the effectiveness of the best-known ML-based TCP technique. In addition, we also investigate if the improvement is statistically significant and has a practical value. The answer to this question can reveal whether data balancing techniques are necessary and useful in the TCP context.

### 4.2 Subjects

In a recent paper, Yaraghi et al. [54] conducted a thorough review of ML-based techniques based on 25 datasets, each with a comprehensive set of features (150 features). As far as we know, the datasets are the most comprehensive publicly available datasets, gathered by analyzing over 20,000 five stars open-source Java projects. They ensured that all subjects have an adequate number of failed tests, and a regression testing time higher than 5 minutes (6 to 67 minutes

with a median of 12), both of which are critical for applying and evaluating TCP techniques.

Also, the datasets are representative in terms of source code size (ranging from 61k to 4.56M lines of code with a median of 229k), number of tests in each CI build (ranging from 33 to 4368 with a median of 117), and number of failed builds (6 to 343 with a median of 83). Table 2 highlights the characteristics of the subjects in terms of lines of code, number of (failed) builds, failure rate, number of commits, and average number of regression tests per build. In the table, the first column (*SID*) lists the identifier of the subjects that will be used throughout this section to refer to them.

Further, the datasets contain the most comprehensive set of features, despite the limited number of features in the other datasets. Overall, the features include execution history of tests, source code of tests, coverage of tests, and process and complexity metrics related to the source code of tests and the SUT, which are collected from the source code of tests and SUT, the execution logs of CI builds, etc. Readers are invited to refer to [54] for a detailed description of the features.

### 4.3 Evaluation Metrics

In this work, similar to many TCP studies (e.g., [15, 54]), we use APFD<sub>C</sub> [15] as the evaluation metric to measure the effectiveness of prioritization techniques. APFD<sub>C</sub> is a cost-aware variant of the well-known and widely used APFD [44] metric. APFD only measures the extent to which a certain ranking reveals faults early, and does not take the execution time of test cases into account, which is important, especially in a CI context. Since we use the publicly available datasets, we inherit two assumptions: (1) since fault severity information is not available, we assume all faults have the same severity. (2) since our collected data does not include the mapping of faults and test cases, similar to prior work [13, 17, 42], we assume that each test case failure refers to a distinct fault in the SUT.

APFD<sub>C</sub> of a test case ordering  $T^*$  is calculated as:

$$\text{APFD}_C = \frac{\sum_{i=1}^m (\sum_{j=TF_i}^n t_j - \frac{1}{2} t_{TF_i})}{\sum_{j=1}^n t_j \times m}$$

where  $m$  refers to the total number of faults,  $n$  refers to the total number of test cases in  $T^*$ ,  $TF_i$  refers to the position (starting from 1) of the first failed test case in  $T^*$  that detects the  $i$ th fault, and  $t_j$  refers to the execution time of the  $j$ th test.

Both APFD and APFD<sub>C</sub> can only be computed for builds that contain failures. Thus, here we only report APFD<sub>C</sub> based on failed builds.

### 4.4 Configuration

Data balancing for each subject and training of ML models was conducted using one CPU core (Intel Xeon Gold 6234 CPU @ 3.30GHz) with a memory usage ranging between 1GB to 20GB, depending on the size of the subjects. We used RankLib [14] for training our TCP ML ranking models, and the *Imbalanced-learn* toolbox for the sampling techniques.



**Table 2: The statistics of the 25 carefully selected subjects. The statistics include source lines of code (SLOC), number of commits, length of the time period between the first and last build, number of all builds and failed builds with the failure rate, average number of test cases per build, and average execution time of all test cases per build [54].**

| $S_{ID}$ | Subject                            | SLOC  | Java SLOC | # Commits | Time period (months) | # Builds | # Failed Builds | Failure Rate (%) | Avg. # TC/Build | Avg. Test Time (min) |
|----------|------------------------------------|-------|-----------|-----------|----------------------|----------|-----------------|------------------|-----------------|----------------------|
| $S_1$    | JMRI/JMRI                          | 4.56M | 1.05M     | 69.3k     | 5                    | 1,481    | 65              | 4                | 4364            | 25                   |
| $S_2$    | apache/airavata                    | 1.46M | 731k      | 10.0k     | 15                   | 236      | 83              | 35               | 49              | 6                    |
| $S_3$    | SonarSource/sonarqube              | 899k  | 398k      | 31.8k     | 18                   | 4,286    | 230             | 5                | 1309            | 6                    |
| $S_4$    | apache/sling                       | 695k  | 388k      | 47.4k     | 7                    | 1,403    | 343             | 24               | 189             | 6                    |
| $S_5$    | camunda/camunda-bpm-platform       | 653k  | 395k      | 20.6k     | 34                   | 822      | 125             | 15               | 569             | 23                   |
| $S_6$    | facebook/buck                      | 586k  | 384k      | 26.3k     | 10                   | 846      | 130             | 15               | 663             | 26                   |
| $S_7$    | apache/shardingsphere              | 422k  | 165k      | 29.6k     | 7                    | 1,049    | 123             | 11               | 789             | 17                   |
| $S_8$    | b2ihealthcare/snow-owl             | 373k  | 212k      | 13.4k     | 2                    | 277      | 21              | 7                | 46              | 10                   |
| $S_9$    | Angel-ML/angel                     | 336k  | 204k      | 3.0k      | 23                   | 308      | 124             | 40               | 33              | 20                   |
| $S_{10}$ | apache/logging-log4j2              | 313k  | 166k      | 12.7k     | 13                   | 441      | 122             | 27               | 544             | 8                    |
| $S_{11}$ | eclipse/jetty.project              | 282k  | 199k      | 25.0k     | 2                    | 192      | 89              | 46               | 137             | 6                    |
| $S_{12}$ | optimatika/ojAlgo                  | 246k  | 84k       | 1.6k      | 22                   | 254      | 72              | 28               | 136             | 9                    |
| $S_{13}$ | yamcs/Yamcs                        | 229k  | 123k      | 5.6k      | 24                   | 504      | 61              | 12               | 114             | 6                    |
| $S_{14}$ | eclipse/steady                     | 221k  | 98k       | 2.0k      | 13                   | 675      | 51              | 7                | 81              | 7                    |
| $S_{15}$ | Graylog2/graylog2-server           | 182k  | 85k       | 22.3k     | 53                   | 3,668    | 124             | 3                | 110             | 20                   |
| $S_{16}$ | CompEvol/beast2                    | 159k  | 83k       | 3.0k      | 85                   | 415      | 115             | 27               | 65              | 6                    |
| $S_{17}$ | EMResearch/EvoMaster               | 158k  | 25k       | 4.1k      | 7                    | 583      | 109             | 18               | 100             | 12                   |
| $S_{18}$ | apache/rocketmq                    | 135k  | 100k      | 2.0k      | 16                   | 536      | 56              | 10               | 182             | 17                   |
| $S_{19}$ | zolyfarkas/spf4j                   | 125k  | 79k       | 32.6k     | 37                   | 587      | 180             | 30               | 116             | 7                    |
| $S_{20}$ | spring-cloud/spring-cloud-dataflow | 104k  | 54k       | 3.5k      | 9                    | 408      | 19              | 4                | 115             | 17                   |
| $S_{21}$ | cantaloupe-project/cantaloupe      | 98k   | 77k       | 4.5k      | 29                   | 450      | 65              | 14               | 148             | 11                   |
| $S_{22}$ | thinkaurelius/titan                | 85k   | 40k       | 5.1k      | 25                   | 384      | 41              | 10               | 45              | 48                   |
| $S_{23}$ | apache/curator                     | 84k   | 58k       | 3.1k      | 21                   | 517      | 65              | 12               | 115             | 67                   |
| $S_{24}$ | jcabi/jcabi-github                 | 61k   | 32k       | 2.8k      | 29                   | 788      | 6               | < 0.01           | 176             | 14                   |
| $S_{25}$ | eclipse/paho.mqtt.java             | 61k   | 34k       | 1.0k      | 16                   | 378      | 77              | 20               | 37              | 15                   |

## 4.5 Experiment Design, Results, and Discussion

**4.5.1 Application of data balancing techniques for TCP (RQ1).** In the following, we assume that the builds of each subject are assigned a unique ID incrementally, according to their time of occurrence.

**Selection of the ML technique** Recent studies [5, 9, 54] indicated that a pairwise ranking approach performs better than list-wise and point-wise ranking in the TCP context. Also, Yaraghi et al. [54] evaluated six ML ranking models based on 25 subjects. The ML ranking models include, MART, LambdaMART (LMART) [53], Random Forest (RF) [10], RankBoost [19], ListNet [11], and Coordinate Ascent (CA) [38]. According to their study, the RF ranking model performed significantly better than any of the other models. Thus, we used RF for all the experiments in this work. Since our study is primarily concerned with assessing the impact of data balancing techniques on TCP effectiveness, experiments using the best-known ML-based TCP is a wise decision. In addition, we used the same hyper-parameters as those reported by Yaraghi et al. [54].

**Selection of the data balancing technique** We explored 19 sampling methods that are supported by the *Imbalanced-learn* toolbox [34] from the three groups of the data balancing techniques, as listed in Table 1. All techniques except cleaning under-sampling are evaluated using three ratios of 1, 2/3, and 1/3 of failed tests over passed tests. Experimenting using three ratios rather than one accounts for the impact of the ratio on the effectiveness of the sampling techniques.

To answer RQ1, we included the latest 51 failed builds (1..51) of each dataset. Our decision for training the models only based on failed builds is based on the results of a recent study by Elsner et al. [17] that showed that training ML-based TCP models by including builds with no failure does not improve the effectiveness

of the TCP models. Thus, including such builds only increases the training time without any benefit. Also, including the 51 last builds is sufficient to run enough experiments to account for the randomness, and come to statistically convincing conclusions. For each subject, we conducted the following steps for each of the sampling techniques that are listed in Table 1:

- For each of the 51 builds except the last build (build 51), we applied the sampling technique with the ratio of 1, and created the sampled training set.
- We trained 50 RF models based on the sampled training set. The trained model for build  $n$ , contains all the data from build 1 to build  $n - 1$ .
- We tested each of the trained models on the consecutive build, e.g., the model that is trained from build 1.. $n$ , is tested on build  $n + 1$ . Note that for the test, we used the original data, whereas the sampled data was only used for the training.

We repeated the same experiment as mentioned above with the ratios of 2/3, and 1/3 for all sampling techniques, except the cleaning under-sampling technique. Overall, we trained 150 models per subject for each of the 12 data balancing techniques with configurable ratio support (45,000 models), and 50 models per each subject for each of cleaning under-sampling techniques (8,750 models).

**RQ1.1** Which combinations of data balancing techniques and balancing ratio perform best?

Tables 3 and 4 show the average APFD<sub>C</sub> for the 25 subjects, based on different balancing techniques and sampling ratio configurations. The columns and rows of both tables (except those in the last two rows and the last two columns of Table 4), correspond to balancing techniques and subjects, respectively. For techniques with configurable sampling ratios, each column contains three sub-columns.

**Table 3: Average APFD<sub>C</sub> of the best ML-based TCP technique trained using the sampled dataset using over-sampling and hybrid techniques with ratios of 1/3, 2/3, and 1.**

| $S_{ID}$ | RO   |      |      | SM   |      |      | ADASYN |      |      | BSM  |      |      | KSM  |      |      | SSM  |      |      | SME  |      |      | SMT  |      |      |
|----------|------|------|------|------|------|------|--------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
|          | 1/3  | 2/3  | 1    | 1/3  | 2/3  | 1    | 1/3    | 2/3  | 1    | 1/3  | 2/3  | 1    | 1/3  | 2/3  | 1    | 1/3  | 2/3  | 1    | 1/3  | 2/3  | 1    | 1/3  | 2/3  | 1    |
| $S_1$    | 0.88 | 0.88 | 0.87 | 0.85 | 0.86 | 0.85 | 0.85   | 0.86 | 0.85 | 0.84 | 0.85 | 0.86 | 0.80 | 0.81 | 0.81 | 0.82 | 0.82 | 0.83 | 0.86 | 0.85 | 0.87 | 0.85 | 0.86 | 0.87 |
| $S_2$    | 0.85 | 0.85 | 0.84 | 0.84 | 0.84 | 0.84 | 0.85   | 0.84 | 0.84 | 0.84 | 0.84 | 0.84 | 0.84 | 0.84 | 0.84 | 0.84 | 0.84 | 0.84 | 0.84 | 0.84 | 0.85 | 0.84 | 0.84 | 0.84 |
| $S_3$    | 0.74 | 0.74 | 0.72 | 0.73 | 0.73 | 0.72 | 0.72   | 0.73 | 0.72 | 0.73 | 0.73 | 0.72 | 0.74 | 0.75 | 0.75 | 0.60 | 0.59 | 0.58 | 0.71 | 0.71 | 0.69 | 0.73 | 0.73 | 0.71 |
| $S_4$    | 0.91 | 0.91 | 0.92 | 0.78 | 0.78 | 0.79 | 0.75   | 0.75 | 0.76 | 0.76 | 0.75 | 0.77 | 0.75 | 0.75 | 0.76 | 0.75 | 0.75 | 0.75 | 0.76 | 0.76 | 0.75 | 0.75 | 0.76 | 0.75 |
| $S_5$    | 0.93 | 0.92 | 0.93 | 0.91 | 0.92 | 0.93 | 0.91   | 0.92 | 0.92 | 0.91 | 0.91 | 0.92 | 0.91 | 0.90 | 0.90 | 0.69 | 0.58 | 0.57 | 0.90 | 0.90 | 0.90 | 0.91 | 0.91 | 0.91 |
| $S_6$    | 0.88 | 0.88 | 0.87 | 0.87 | 0.86 | 0.85 | 0.88   | 0.88 | 0.89 | 0.86 | 0.87 | 0.85 | 0.88 | 0.88 | 0.88 | 0.86 | 0.87 | 0.86 | 0.87 | 0.87 | 0.88 | 0.87 | 0.87 | 0.88 |
| $S_7$    | 0.98 | 0.99 | 0.99 | 0.99 | 0.99 | 0.99 | 0.99   | 0.99 | 0.99 | 0.98 | 0.99 | 0.99 | 0.98 | 0.98 | 0.98 | 0.98 | 0.99 | 0.99 | 0.99 | 0.99 | 0.99 | 0.99 | 0.99 | 0.99 |
| $S_8$    | 0.66 | 0.67 | 0.67 | 0.65 | 0.66 | 0.66 | 0.66   | 0.65 | 0.65 | 0.65 | 0.66 | 0.66 | 0.65 | 0.66 | 0.66 | 0.65 | 0.65 | 0.65 | 0.65 | 0.65 | 0.65 | 0.65 | 0.65 | 0.65 |
| $S_9$    | 0.81 | 0.82 | 0.82 | 0.81 | 0.82 | 0.82 | 0.82   | 0.81 | 0.82 | 0.81 | 0.82 | 0.82 | 0.82 | 0.82 | 0.81 | 0.81 | 0.81 | 0.82 | 0.80 | 0.71 | 0.72 | 0.81 | 0.81 | 0.82 |
| $S_{10}$ | 0.95 | 0.95 | 0.95 | 0.94 | 0.94 | 0.94 | 0.93   | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 |
| $S_{11}$ | 0.95 | 0.95 | 0.95 | 0.94 | 0.93 | 0.94 | 0.94   | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.94 | 0.93 |
| $S_{12}$ | 0.86 | 0.86 | 0.86 | 0.84 | 0.85 | 0.85 | 0.86   | 0.84 | 0.85 | 0.84 | 0.84 | 0.85 | 0.85 | 0.85 | 0.85 | 0.86 | 0.85 | 0.85 | 0.84 | 0.85 | 0.84 | 0.85 | 0.85 | 0.84 |
| $S_{13}$ | 0.73 | 0.74 | 0.75 | 0.75 | 0.76 | 0.76 | 0.74   | 0.75 | 0.76 | 0.74 | 0.75 | 0.75 | 0.75 | 0.75 | 0.75 | 0.72 | 0.71 | 0.74 | 0.74 | 0.74 | 0.74 | 0.74 | 0.74 | 0.75 |
| $S_{14}$ | 0.92 | 0.91 | 0.92 | 0.92 | 0.93 | 0.92 | 0.93   | 0.92 | 0.93 | 0.93 | 0.93 | 0.93 | 0.92 | 0.93 | 0.92 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 |
| $S_{15}$ | 0.98 | 0.98 | 0.98 | 0.98 | 0.98 | 0.98 | 0.98   | 0.98 | 0.98 | 0.98 | 0.98 | 0.98 | 0.97 | 0.98 | 0.98 | 0.98 | 0.98 | 0.98 | 0.98 | 0.98 | 0.98 | 0.98 | 0.98 | 0.98 |
| $S_{16}$ | 0.75 | 0.75 | 0.77 | 0.77 | 0.78 | 0.78 | 0.78   | 0.78 | 0.78 | 0.78 | 0.78 | 0.77 | 0.78 | 0.78 | 0.78 | 0.76 | 0.74 | 0.72 | 0.77 | 0.78 | 0.78 | 0.78 | 0.78 | 0.77 |
| $S_{17}$ | 0.76 | 0.77 | 0.79 | 0.79 | 0.78 | 0.78 | 0.78   | 0.79 | 0.78 | 0.77 | 0.79 | 0.78 | 0.77 | 0.78 | 0.77 | 0.77 | 0.77 | 0.74 | 0.78 | 0.79 | 0.77 | 0.78 | 0.78 | 0.78 |
| $S_{18}$ | 0.84 | 0.84 | 0.84 | 0.84 | 0.86 | 0.87 | 0.84   | 0.84 | 0.86 | 0.84 | 0.84 | 0.86 | 0.83 | 0.81 | 0.81 | 0.84 | 0.84 | 0.84 | 0.78 | 0.83 | 0.84 | 0.83 | 0.84 | 0.84 |
| $S_{19}$ | 0.93 | 0.92 | 0.92 | 0.93 | 0.93 | 0.93 | 0.93   | 0.93 | 0.93 | 0.93 | 0.93 | 0.93 | 0.92 | 0.93 | 0.93 | 0.93 | 0.93 | 0.92 | 0.93 | 0.92 | 0.93 | 0.93 | 0.93 | 0.93 |
| $S_{20}$ | 0.93 | 0.92 | 0.93 | 0.92 | 0.92 | 0.93 | 0.92   | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 | 0.92 |
| $S_{21}$ | 0.80 | 0.80 | 0.81 | 0.81 | 0.81 | 0.82 | 0.81   | 0.81 | 0.80 | 0.80 | 0.80 | 0.80 | 0.80 | 0.80 | 0.80 | 0.79 | 0.78 | 0.78 | 0.80 | 0.80 | 0.80 | 0.80 | 0.80 | 0.81 |
| $S_{22}$ | 0.73 | 0.75 | 0.75 | 0.72 | 0.72 | 0.74 | 0.73   | 0.72 | 0.72 | 0.73 | 0.73 | 0.73 | 0.73 | 0.73 | 0.73 | 0.73 | 0.72 | 0.73 | 0.72 | 0.73 | 0.72 | 0.74 | 0.72 | 0.73 |
| $S_{23}$ | 0.84 | 0.85 | 0.86 | 0.86 | 0.85 | 0.86 | 0.85   | 0.85 | 0.86 | 0.85 | 0.85 | 0.85 | 0.85 | 0.85 | 0.86 | 0.85 | 0.85 | 0.84 | 0.85 | 0.85 | 0.85 | 0.85 | 0.85 | 0.85 |
| $S_{24}$ | 0.93 | 0.95 | 0.96 | 0.95 | 0.96 | 0.96 | 0.95   | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 | 0.95 |
| $S_{25}$ | 0.66 | 0.66 | 0.66 | 0.68 | 0.70 | 0.70 | 0.69   | 0.69 | 0.68 | 0.68 | 0.69 | 0.69 | 0.70 | 0.69 | 0.70 | 0.70 | 0.69 | 0.71 | 0.70 | 0.70 | 0.69 | 0.70 | 0.69 | 0.68 |
| Best     | 11   | 10   | 13   | 10   | 11   | 12   | 9      | 9    | 9    | 9    | 9    | 9    | 9    | 9    | 9    | 9    | 8    | 8    | 9    | 9    | 9    | 9    | 9    | 9    |
| Worst    | 0    | 0    | 0    | 0    | 0    | 0    | 0      | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    |

The APFD<sub>C</sub> values for the best configuration(s) for each subject are displayed in **bold**, with the exception of subjects *S<sub>2</sub>*, *S<sub>6</sub>* and *S<sub>23</sub>*, where the differences between configurations' results are not statistically significant. We determine the best configurations by calculating the Relative Performance Index (*RPI*) for each configuration, which indicates the number of configurations with a significantly lower average in terms of APFD<sub>C</sub>. The *RPI* can be the same between two or more configurations when there is no statistical difference between them. For instance, for subject *S<sub>4</sub>*, the best configurations are Random Over-sampling (RO) with the ratio of 2/3 and 1, since their *RPI* are equal, and significantly higher than other configurations.

To check whether there are significant differences across configurations for a given subject, we used Welch's ANOVA [51] to compare all configurations across all builds, with one APFD<sub>C</sub> value per each build. Then we performed the Games-Howell post-hoc test [20] to compare each pair of configurations. The significance level is set to 0.05, and therefore any difference with *p*-value  $\leq 0.05$  is considered significant. We used Welch's ANOVA rather than one-way ANOVA, because the variances in results across configurations are not equal. Also, we rely on the Games-Howell post-hoc test due to its compatibility with Welch's ANOVA (i.e., no assumption about equal variance). This post-hoc test also addresses the usual problems related to repeated testing when performing multiple comparisons (increased type-I error) [29].

As shown in Tables 3 and 4, multiple configurations (highlighted with the bold font) perform best for some of the subjects, but there is no single configuration that performs best across all subjects. In Tables 3 and 4, the last two rows (*Best* and *Worst*), indicate the

number of times where a particular configuration has provided the best and worst results for a subject respectively. The top three configurations across all subjects are RO with ratio 1, SMOTE (SM) with ratio 1, and Instance Hardness Threshold (*IHT*) with ratio 1. Also, the best configurations for all subjects except *S<sub>25</sub>* include either RO or *IHT*. This suggests that application of either RO or *IHT* can lead to the best results across all subjects except for *S<sub>25</sub>*. In addition, the Cluster Centeriod (*CC*) configuration performed the worst across all subjects, and never provided the best results. This suggests that the *CC* is not a effective technique in the context of TCP.

Also, for each dataset, we measured the effect size of the differences between configuration pairs based on Common Language Effect Size (CLE) [4, 37]. The CLE estimates the probability of a randomly sampled score from one population being greater than a randomly sampled score from another population. As shown in the last column (*CLE*) of Table 4, CLE values between a worst case and a best case are above 74% for all subjects except *S<sub>2</sub>*, *S<sub>6</sub>* and *S<sub>23</sub>*. Results, therefore, show the importance of selecting one of the best configurations.

**RQ1.2** Does the balancing ratio impact the effectiveness of the balancing techniques?

We conducted a Welch's ANOVA analysis and Games-Howell post-hoc tests on each of the 12 balancing techniques with the ratio configurations, to determine whether the ratio affects the effectiveness of balancing techniques. For each analysis, we divided the results of each technique across all subjects into three groups, corresponding to 1/3, 2/3, and 1. In 10 out of 12 techniques, no statistically

**Table 4: Average APFD<sub>C</sub> of the best ML-based TCP technique using under-sampling techniques with sampling ratios of 1/3, 2/3, and 1.**

| $S_{ID}$ | RU          |             |             | CC       |          |           | NM          |             |             | IHT         |             |             | ENN         | REN         | AKN         | CNN         | OSS         | NCR         | TL          | Statistics |      |
|----------|-------------|-------------|-------------|----------|----------|-----------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|------------|------|
|          | 1/3         | 2/3         | 1           | 1/3      | 2/3      | 1         | 1/3         | 2/3         | 1           | 1/3         | 2/3         | 1           |             |             |             |             |             |             |             | p-val      | CLE  |
| $S_1$    | 0.57        | 0.56        | 0.56        | 0.53     | 0.55     | 0.57      | 0.75        | 0.75        | 0.75        | 0.82        | 0.81        | 0.81        | 0.81        | 0.81        | 0.81        | 0.61        | 0.70        | 0.80        | 0.81        | 0.000      | 0.84 |
| $S_2$    | 0.83        | 0.83        | 0.83        | 0.83     | 0.83     | 0.83      | 0.83        | 0.83        | 0.83        | 0.83        | 0.83        | 0.83        | 0.84        | 0.84        | 0.84        | 0.83        | 0.84        | 0.84        | 0.84        | 0.999      | 0.50 |
| $S_3$    | 0.47        | 0.40        | 0.40        | 0.52     | 0.40     | 0.34      | 0.71        | 0.70        | 0.72        | <b>0.75</b> | <b>0.75</b> | <b>0.74</b> | <b>0.74</b> | <b>0.74</b> | 0.74        | 0.72        | <b>0.76</b> | <b>0.74</b> | <b>0.75</b> | 0.000      | 0.84 |
| $S_4$    | 0.91        | 0.81        | 0.85        | 0.64     | 0.44     | 0.65      | 0.77        | 0.78        | 0.78        | 0.79        | 0.78        | 0.79        | 0.76        | 0.77        | 0.77        | 0.86        | 0.69        | 0.78        | 0.75        | 0.000      | 0.98 |
| $S_5$    | 0.54        | 0.48        | 0.45        | 0.67     | 0.57     | 0.54      | 0.69        | 0.68        | 0.68        | 0.80        | 0.80        | 0.80        | 0.90        | 0.90        | 0.90        | 0.55        | 0.84        | 0.90        | 0.90        | 0.000      | 0.96 |
| $S_6$    | 0.84        | 0.78        | 0.81        | 0.74     | 0.77     | 0.81      | 0.88        | 0.88        | 0.88        | 0.86        | 0.86        | 0.85        | 0.88        | 0.87        | 0.87        | 0.88        | 0.89        | 0.88        | 0.88        | 0.536      | 0.53 |
| $S_7$    | <b>0.99</b> | 0.98        | 0.97        | 0.95     | 0.90     | 0.81      | 0.52        | 0.35        | 0.23        | 0.98        | 0.98        | <b>0.98</b> | 0.98        | 0.98        | 0.98        | <b>0.98</b> | 0.98        | 0.98        | 0.98        | 0.000      | 1.00 |
| $S_8$    | 0.59        | 0.41        | 0.40        | 0.46     | 0.38     | 0.38      | <b>0.65</b> | <b>0.66</b> | <b>0.66</b> | <b>0.69</b> | <b>0.69</b> | <b>0.69</b> | <b>0.66</b> | <b>0.65</b> | <b>0.65</b> | 0.44        | 0.37        | <b>0.66</b> | <b>0.65</b> | 0.000      | 0.86 |
| $S_9$    | 0.82        | 0.78        | 0.80        | 0.75     | 0.74     | 0.76      | 0.78        | 0.75        | 0.73        | 0.82        | 0.81        | 0.81        | 0.71        | 0.73        | 0.68        | <b>0.83</b> | 0.70        | 0.75        | 0.80        | 0.000      | 0.76 |
| $S_{10}$ | <b>0.95</b> | <b>0.89</b> | <b>0.95</b> | 0.68     | 0.71     | 0.64      | <b>0.93</b> | <b>0.93</b> | <b>0.93</b> | <b>0.94</b> | <b>0.94</b> | <b>0.95</b> | <b>0.93</b> | <b>0.93</b> | <b>0.93</b> | <b>0.94</b> | <b>0.86</b> | <b>0.93</b> | <b>0.93</b> | 0.000      | 0.95 |
| $S_{11}$ | 0.73        | 0.39        | 0.48        | 0.22     | 0.10     | 0.06      | <b>0.94</b> | <b>0.94</b> | <b>0.94</b> | <b>0.92</b> | <b>0.93</b> | <b>0.93</b> | <b>0.94</b> | <b>0.93</b> | <b>0.93</b> | <b>0.94</b> | <b>0.92</b> | <b>0.94</b> | <b>0.94</b> | 0.000      | 1.00 |
| $S_{12}$ | 0.77        | 0.58        | 0.75        | 0.76     | 0.76     | 0.78      | 0.84        | 0.84        | 0.84        | 0.85        | 0.85        | 0.85        | 0.80        | 0.81        | 0.82        | 0.77        | 0.74        | 0.83        | 0.85        | 0.000      | 0.80 |
| $S_{13}$ | 0.64        | 0.61        | 0.60        | 0.61     | 0.57     | 0.56      | 0.68        | 0.67        | 0.66        | 0.75        | <b>0.76</b> | <b>0.75</b> | 0.75        | 0.74        | 0.74        | 0.71        | 0.67        | 0.75        | 0.74        | 0.000      | 0.74 |
| $S_{14}$ | 0.89        | 0.77        | 0.82        | 0.84     | 0.75     | 0.78      | 0.90        | 0.90        | 0.89        | <b>0.94</b> | <b>0.93</b> | <b>0.93</b> | 0.92        | 0.93        | 0.92        | 0.90        | 0.86        | 0.93        | 0.92        | 0.000      | 0.78 |
| $S_{15}$ | 0.97        | 0.93        | 0.95        | 0.93     | 0.93     | 0.97      | 0.98        | 0.97        | 0.95        | 0.98        | 0.98        | 0.98        | 0.98        | 0.98        | 0.98        | <b>0.99</b> | 0.95        | 0.98        | 0.98        | 0.000      | 0.77 |
| $S_{16}$ | 0.60        | 0.49        | 0.48        | 0.55     | 0.44     | 0.36      | 0.74        | 0.73        | 0.73        | <b>0.79</b> | <b>0.78</b> | <b>0.80</b> | <b>0.79</b> | <b>0.79</b> | <b>0.80</b> | 0.76        | <b>0.79</b> | <b>0.80</b> | <b>0.78</b> | 0.000      | 0.90 |
| $S_{17}$ | 0.51        | 0.46        | 0.45        | 0.40     | 0.33     | 0.31      | <b>0.73</b> | <b>0.73</b> | <b>0.74</b> | <b>0.77</b> | <b>0.78</b> | <b>0.78</b> | <b>0.77</b> | <b>0.77</b> | <b>0.76</b> | <b>0.68</b> | 0.66        | <b>0.79</b> | <b>0.78</b> | 0.000      | 0.92 |
| $S_{18}$ | 0.65        | 0.62        | 0.60        | 0.83     | 0.63     | 0.62      | 0.78        | 0.79        | 0.80        | 0.81        | 0.81        | 0.80        | 0.82        | 0.73        | 0.76        | 0.82        | 0.61        | 0.76        | 0.80        | 0.000      | 0.79 |
| $S_{19}$ | <b>0.91</b> | 0.54        | 0.72        | 0.61     | 0.38     | 0.30      | <b>0.92</b> | <b>0.92</b> | <b>0.92</b> | <b>0.93</b> | <b>0.93</b> | <b>0.93</b> | 0.84        | 0.86        | 0.86        | <b>0.91</b> | 0.63        | 0.81        | <b>0.93</b> | 0.000      | 0.95 |
| $S_{20}$ | 0.68        | 0.51        | 0.58        | 0.49     | 0.45     | 0.51      | <b>0.92</b> | <b>0.92</b> | <b>0.92</b> | <b>0.92</b> | <b>0.92</b> | <b>0.92</b> | <b>0.92</b> | <b>0.92</b> | <b>0.92</b> | 0.56        | 0.76        | <b>0.92</b> | <b>0.92</b> | 0.000      | 0.94 |
| $S_{21}$ | 0.61        | 0.51        | 0.47        | 0.64     | 0.55     | 0.52      | 0.74        | 0.74        | 0.73        | 0.79        | 0.79        | 0.79        | 0.80        | 0.79        | 0.80        | 0.78        | 0.79        | 0.80        | 0.80        | 0.000      | 0.84 |
| $S_{22}$ | 0.65        | 0.58        | 0.56        | 0.61     | 0.56     | 0.55      | 0.65        | 0.59        | 0.59        | 0.72        | 0.70        | 0.70        | 0.73        | 0.74        | 0.73        | 0.68        | 0.72        | 0.73        | 0.74        | 0.000      | 0.74 |
| $S_{23}$ | 0.90        | 0.86        | 0.86        | 0.80     | 0.75     | 0.77      | 0.82        | 0.83        | 0.82        | 0.84        | 0.85        | 0.85        | 0.82        | 0.83        | 0.83        | 0.91        | 0.79        | 0.85        | 0.85        | 0.308      | 0.45 |
| $S_{24}$ | <b>0.86</b> | 0.44        | 0.44        | 0.52     | 0.44     | 0.39      | 0.51        | 0.48        | 0.48        | <b>0.95</b> | <b>0.95</b> | <b>0.95</b> | <b>0.95</b> | <b>0.95</b> | <b>0.95</b> | <b>0.94</b> | <b>0.95</b> | <b>0.95</b> | <b>0.95</b> | 0.000      | 0.81 |
| $S_{25}$ | 0.42        | 0.37        | 0.48        | 0.75     | 0.54     | 0.61      | 0.71        | 0.70        | 0.71        | 0.68        | 0.69        | 0.69        | 0.62        | 0.48        | 0.50        | 0.75        | <b>0.79</b> | 0.70        | 0.66        | 0.000      | 0.89 |
| Best     | <b>4</b>    | <b>1</b>    | <b>1</b>    | <b>0</b> | <b>0</b> | <b>0</b>  | <b>6</b>    | <b>6</b>    | <b>6</b>    | <b>10</b>   | <b>11</b>   | <b>12</b>   | <b>8</b>    | <b>8</b>    | <b>7</b>    | <b>8</b>    | <b>6</b>    | <b>8</b>    | <b>9</b>    |            |      |
| Worst    | <b>0</b>    | <b>7</b>    | <b>5</b>    | <b>5</b> | <b>6</b> | <b>12</b> | <b>1</b>    | <b>1</b>    | <b>2</b>    | <b>0</b>    | <b>0</b>    | <b>0</b>    | <b>0</b>    | <b>0</b>    | <b>1</b>    | <b>1</b>    | <b>2</b>    | <b>0</b>    | <b>0</b>    |            |      |

significant differences exist between the results of techniques with different ratios. The results differ statistically when different ratios are used for two techniques *RU* and *CC*. Both techniques perform best with a ratio of 1/3.

The *RU* and *CC* techniques do not perform well in general, as shown in 4. In addition, the ratio does not play a significant role for best techniques (*RO*, *SM*, *IHT*); therefore, it is safe to use the default ratio (i.e., 1) when applying balancing techniques.

**RQ1.3** Which group (under-sampling, over-sampling, and hybrid) of the data balancing techniques perform best across all subjects?

We conducted a Welch's ANOVA analysis and Games-Howell post-hoc test based on results with the ratio 1, for all subjects divided into three groups: under-sampling, over-sampling, and hybrid. As a result of the analysis, results obtained using over-sampling, and hybrid techniques are not statistically different. However, they outperform under-sampling techniques with the p-values and CLE 0.000, 0.57 respectively. We argue that this is attributed to low results for some under-sampling techniques, specifically *CC* and *RU*. However, we still cannot recommend ruling out the use of under-sampling techniques in the TCP context, as some under-sampling techniques provide better results for five subjects than over-sampling techniques. The results of hybrid techniques are not statistically different from the results of over-sampling; however, none of them can be considered as the single best technique for a

specific subject. Thus, it is safe to rule out hybrid techniques in the TCP context.

**RQ1:** *RO*, *SM*, and *IHT* provide the best results in the TCP context. Conversely, *CC* and *RU* provide the worst results. Also, the sampling ratio has no significant impact on the effectiveness of balancing techniques in the TCP context.

**4.5.2 (RQ2). Baseline.** As discussed, we used the recent work by Yaraghi et al. [54] as the baseline of our work. They have reported that RF Ranking models outperform other ranking models, and heuristics in the TCP context. To answer *RQ2*, we replicated their experiments based on the original subjects, without sampling, using the same procedure described for *RQ1* based on the last 51 failed builds. The column *No Sam.* of Table 5 shows the results of the TCP in terms of APFD<sub>C</sub>, using the RF ranking models trained without balancing training data (baseline). To determine whether or not the balancing techniques improve the effectiveness of the TCP techniques, we compared the baseline with (1) the *RO* with ratio 1 that provides the best results across all subjects, and (2) the best technique per each subject with the highest RPI. Whenever more than one technique has the highest RPI for one subject, the technique with the highest average is selected.

**RQ2.1** How does the application of the best data balancing technique across all subjects impact the TCP effectiveness?

**Table 5: Comparison of the best ML-based TCP technique results across all subjects using training data (1) without sampling, (2) sampling using Random Over-sampling (RO) (the best technique across all subjects), and (3) sampling using the best techniques per each subject.**

| $S_{ID}$ | No Sam.           | Best Overall      |             |        | Best Per Subject  |             |       |
|----------|-------------------|-------------------|-------------|--------|-------------------|-------------|-------|
|          | APFD <sub>C</sub> | APFD <sub>C</sub> | Imp.        | p-val. | APFD <sub>C</sub> | Imp.        | p-val |
| $S_1$    | 0.77±0.25         | 0.88±0.14         | 0.11        | 0.017  | 0.88±0.14         | 0.11        | 0.007 |
| $S_2$    | 0.84±0.04         | 0.85±0.04         | 0.0         | 0.941  | 0.85±0.04         | 0.0         | 0.741 |
| $S_3$    | 0.75±0.25         | 0.72±0.29         | -0.03       | 0.753  | 0.76±0.24         | 0.01        | 0.857 |
| $S_4$    | 0.79±0.2          | 0.92±0.14         | 0.13        | 0.0    | 0.92±0.14         | 0.13        | 0.0   |
| $S_5$    | 0.9±0.12          | 0.93±0.12         | 0.03        | 0.331  | 0.93±0.12         | 0.03        | 0.22  |
| $S_6$    | 0.88±0.23         | 0.87±0.23         | -0.01       | 0.924  | 0.89±0.21         | 0.01        | 0.804 |
| $S_7$    | 0.91±0.17         | 0.99±0.02         | 0.08        | 0.006  | 0.99±0.02         | 0.08        | 0.002 |
| $S_8$    | 0.78±0.14         | 0.67±0.13         | -0.11       | 0.017  | 0.69±0.11         | -0.09       | 0.016 |
| $S_9$    | 0.81±0.2          | 0.82±0.19         | 0.01        | 0.857  | 0.83±0.19         | 0.02        | 0.579 |
| $S_{10}$ | 0.94±0.14         | 0.95±0.12         | 0.02        | 0.532  | 0.95±0.12         | 0.02        | 0.532 |
| $S_{11}$ | 0.86±0.26         | 0.95±0.12         | 0.09        | 0.039  | 0.95±0.12         | 0.09        | 0.039 |
| $S_{12}$ | 0.85±0.16         | 0.86±0.12         | 0.01        | 0.826  | 0.86±0.12         | 0.01        | 0.823 |
| $S_{13}$ | 0.73±0.24         | 0.74±0.22         | 0.01        | 0.891  | 0.76±0.23         | 0.02        | 0.633 |
| $S_{14}$ | 0.92±0.16         | 0.92±0.17         | -0.01       | 0.755  | 0.94±0.11         | 0.01        | 0.667 |
| $S_{15}$ | 0.98±0.05         | 0.98±0.03         | 0.01        | 0.08   | 0.99±0.01         | 0.01        | 0.053 |
| $S_{16}$ | 0.78±0.22         | 0.77±0.22         | -0.01       | 0.696  | 0.8±0.21          | 0.02        | 0.561 |
| $S_{17}$ | 0.77±0.18         | 0.79±0.16         | 0.02        | 0.754  | 0.79±0.16         | 0.02        | 0.464 |
| $S_{18}$ | 0.81±0.24         | 0.84±0.21         | 0.03        | 0.446  | 0.87±0.18         | 0.05        | 0.204 |
| $S_{19}$ | 0.88±0.12         | 0.92±0.09         | 0.04        | 0.027  | 0.93±0.07         | 0.06        | 0.007 |
| $S_{20}$ | 0.78±0.24         | 0.93±0.12         | 0.14        | 0.027  | 0.93±0.12         | 0.14        | 0.027 |
| $S_{21}$ | 0.77±0.23         | 0.81±0.2          | 0.04        | 0.505  | 0.82±0.2          | 0.05        | 0.264 |
| $S_{22}$ | 0.66±0.29         | 0.75±0.24         | 0.09        | 0.116  | 0.75±0.24         | 0.09        | 0.116 |
| $S_{23}$ | 0.85±0.15         | 0.86±0.15         | 0.0         | 0.122  | 0.91±0.14         | 0.05        | 0.071 |
| $S_{24}$ | 0.56±0.27         | 0.96±0.13         | 0.4         | 0.03   | 0.96±0.12         | 0.4         | 0.03  |
| $S_{25}$ | 0.7±0.31          | 0.66±0.29         | -0.04       | 0.046  | 0.79±0.23         | 0.09        | 0.121 |
| Over.    | <b>0.81</b>       | <b>0.85</b>       | <b>0.04</b> |        | <b>0.87</b>       | <b>0.06</b> |       |

We conducted a Welch’s ANOVA analysis per each subject to compare the results of baseline, and *RO* across all builds, with one APFD<sub>C</sub> value per each build. The *Best Overall* column of Table 5 shows the results of the comparison. *RO* improved the performance of the TCP for 17 out of 25 subjects. The improvement (sub-column Imp. of Table 5) ranges from 0.1 to 0.40 with an average of 0.04, in terms of APFD<sub>C</sub>. The sub-column *p-val* of Table 5 also shows that the improvements are statistically significant for 8 of the subjects (*p-value* ≤ 0.05). Further, the effectiveness of TCP remains unchanged for two subjects, and decreases for six subjects. Two of the decreases are statistically significant. Therefore, we can conclude that, while applying a balancing technique across all subjects can improve the effectiveness of TCP techniques, it may also result in a decrease in the effectiveness.

**RQ2.2** How does the application of the best data balancing technique per each subject impact the TCP effectiveness?

We conducted a Welch’s ANOVA analysis per each subject based on the baseline results, and the best technique for each subject. The *Best Per Subject* column of Table 5 shows the results of the comparison. The results demonstrate that the application of the best balancing techniques to each subject improved the TCP performance for 23 out of 25 subjects. The improvement (sub-column Imp. of Table 5) ranges from 0.1 to 0.40, with an average of 0.06 in terms of APFD<sub>C</sub>. The improvements are significant for 9 subjects. Also, the effectiveness of TCP remained the same for one subject,

and decreased only for one subject, which is statistically significant. Thus, it is safe to conclude that the best technique for each subject that can be determined with preliminary analysis can increase the effectiveness of TCP without posing a significant risk.

**RQ.2:** The application of data balancing techniques can improve the effectiveness of TCP techniques, ranging from 0.01 to 0.40 in terms of APFD<sub>C</sub>. They can, however, decrease the effectiveness of TCP techniques in some cases when applied without investigating on which technique is suitable for each subject.

## 4.6 Threats to Validity

**4.6.1 External Validity.** Over-sampling techniques, specifically random ones, repeatedly replicate a small number of samples, which can lead to over-fitting issues for trained ML models. Under-sampling techniques can also result in under-fitting if many samples are removed from the majority class. These issues are inherent to balancing techniques and are inevitable, particularly when using a balancing ratio of 1. In order to ensure that possible over- or under-fittings did not distort our results, we used separate data for training and testing. Where, the model trained on builds 1..*n* was tested on build *n* + 1. In this context, under- or over-fitting is well-known, and hybrid, SMOTE, and cleaning techniques have been suggested as solutions. In addition, our work is not intended to improve balancing techniques but rather to evaluate and report on their suitability in the context of ML-based TCP.

Our experiment was conducted using publicly available datasets. Our selection of the best-known ML techniques was also based on previous research. There may be flaws in the previous study, and in the available datasets that may impact the validity of our experiments.

**4.6.2 Conclusion Validity.** Our study only uses projects that are developed using Java. Additionally, we investigate only data balancing techniques provided by Imbalanced-learn toolbox. These two issues may threaten the generalization of our conclusion. To mitigate this issue, we do not draw any general conclusions, and we recommend performing a preliminary analysis before applying data balancing techniques.

## 5 CONCLUSION

ML-based TCP techniques often train ML models based on datasets collected from the previous execution of test cases to predict the ranking of test cases based on the probability of their failure and execution time. However, regression test cases rarely fail. This leads to imbalanced datasets for training ML-based TCP techniques that negatively affect the effectiveness of their techniques. This work investigates whether or not data balancing techniques can help with imbalanced datasets in the context of the ML-based TCP. We conducted a thorough empirical analysis by applying 19 state-of-the-art data balancing techniques based on the most comprehensive publicly available datasets. Through training and evaluating of 53,750 of ML-based TCP models of 1,250 builds, using 19 balancing techniques, we conclude that:



- *Random Over-sampling (RO)*, *SMOTE (SM)* [12], and *Instance Hardness Threshold (IHT)* [46] provide the best results in the TCP context. Conversely, *Cluster Centroids (CC)* [55] and *Random Under-sampling (RU)* provide the worst results.
- The sampling ratio does not significantly impact the effectiveness of balancing techniques in the TCP context.
- Data balancing techniques can improve the effectiveness of the best-known ML-based TCP technique for most subjects, with an average of 0.06 in terms of  $APFD_C$ . However, for a few subjects, the effectiveness of TCP is decreased when a specific technique is applied to all subjects.

As demonstrated, data balancing techniques improve the effectiveness of ML-based TCP techniques and result in  $APFD_C$  over 0.90 for 13 (52%) of the studied subjects. However, here is still a need for further optimization and improvement. One possible direction in this context would improve the training datasets by removing the noisy samples caused by flaky and broken tests.

## REFERENCES

- [1] Rafiq Almaghairbe and Marc Roper. 2017. Separating passing and failing test executions by clustering anomalies. *Software Quality Journal* 25, 3 (2017), 803–840.
- [2] Jeff Anderson, Saeed Salem, and Hyunsook Do. 2015. Striving for Failure: An Industrial Case Study about Test Failure Prediction. In *37th IEEE International Conference on Software Engineering*, Vol. 2. IEEE, Florence, Italy, 49–58.
- [3] Jeff Anderson, Saeed Salem, and Hyunsook Do. 2015. Striving for failure: an industrial case study about test failure prediction. In *37th IEEE International Conference on Software Engineering*, Vol. 2. IEEE, Florence, Italy, 49–58.
- [4] Andrea Arcuri and Lionel Briand. 2014. A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. *Software Testing, Verification and Reliability* 24, 3 (2014), 219–250.
- [5] Mojtaba Bagherzadeh, Nafiseh Kahani, and Lionel Briand. 2021. Reinforcement learning for test case prioritization. *IEEE Transactions on Software Engineering* 48, 8 (2021), 1–21. <https://doi.org/10.1109/TSE.2021.3070549>
- [6] Gustavo EAPA Batista, Ana LC Bazzan, Maria Carolina Monard, et al. 2003. Balancing training data for automated annotation of keywords: a case study. In *WOB. IEEE, Macaé, Rio de Janeiro, Brazil*, 10–18.
- [7] Gustavo EAPA Batista, Ronaldo C Prati, and Maria Carolina Monard. 2004. A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD explorations newsletter* 6, 1 (2004), 20–29.
- [8] Moritz Beller, Georgios Gousios, and Andy Zaidman. 2017. Oops, my tests broke the build: an explorative analysis of Travis CI with GitHub. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, Buenos Aires, 356–367.
- [9] Antonia Bertolino, Antonio Guerriero, Breno Miranda, Roberto Pietrantuono, and Stefano Russo. 2020. Learning-to-rank vs ranking-to-learn: strategies for regression testing in continuous integration. In *42nd International Conference on Software Engineering (ICSE)*. ACM/IEEE, Seoul, Republic of Korea, 1–12.
- [10] Leo Breiman. 2001. Random forests. *Machine Learning* 45, 1 (01 Oct 2001), 5–32. <https://doi.org/10.1023/A:1010933404324>
- [11] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. 2007. Learning to rank: from pairwise approach to listwise approach. In *Proceedings of the 24th International Conference on Machine Learning (Corvallis, Oregon, USA) (ICML '07)*. Association for Computing Machinery, New York, NY, USA, 129–136.
- [12] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. 2002. SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research* 16 (2002), 321–357.
- [13] Junjie Chen, Yiling Lou, Lingming Zhang, Jianyi Zhou, Xiaoleng Wang, Dan Hao, and Lu Zhang. 2018. Optimizing test prioritization via test distribution analysis. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, New York, NY, United States, 656–667.
- [14] Dang, Van. 2020. RankLib. <https://sourceforge.net/p/lemur/wiki/RankLib/>.
- [15] Sebastian Elbaum, Alexey Malishevsky, and Gregg Rothermel. 2001. Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of the 23rd International Conference on Software Engineering. ICSE 2001*. IEEE, Toronto, Ontario, Canada, 329–338.
- [16] Sebastian Elbaum, Gregg Rothermel, and John Penix. 2014. Techniques for improving regression testing in continuous integration development environments. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, Hong Kong China, 235–245.
- [17] Daniel Elsner, Florian Hauer, Alexander Pretschner, and Silke Reimer. 2021. Empirically evaluating readily available information for regression test optimization in continuous integration. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, Virtual Denmark, 491–504.
- [18] Martin Fowler and Matthew Foemmel. 2006. Continuous integration.
- [19] Yoav Freund, Raj Iyer, Robert E Schapire, and Yoram Singer. 2003. An efficient boosting algorithm for combining preferences. *Journal of machine learning research* 4, Nov (2003), 933–969.
- [20] Paul A Games and John F Howell. 1976. Pairwise multiple comparison procedures with unequal n's and/or variances: a Monte Carlo study. *Journal of Educational Statistics* 1, 2 (1976), 113–125.
- [21] Hui Han, Wen-Yuan Wang, and Bing-Huan Mao. 2005. Borderline-SMOTE: a new over-sampling method in imbalanced data sets learning. In *International conference on intelligent computing*. Springer, Berlin, Heidelberg, 878–887.
- [22] Peter Hart. 1968. The condensed nearest neighbor rule (corresp.). *IEEE transactions on information theory* 14, 3 (1968), 515–516.
- [23] Haibo He, Yang Bai, Edwardo A Garcia, and Shutao Li. 2008. ADASYN: adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)*. IEEE, Hong Kong, 1322–1328.
- [24] Haibo He and Edwardo A Garcia. 2009. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering* 21, 9 (2009), 1263–1284.
- [25] Kim Herzig, Michaela Greiler, Jacek Czerwinka, and Brendan Murphy. 2015. The art of testing less without sacrificing quality. In *37th IEEE International Conference on Software Engineering*. IEEE, Florence, Italy, 483–493.
- [26] Michael Hilton, Timothy Tunnell, Kai Huang, Darko Marinov, and Danny Dig. 2016. Usage, costs, and benefits of continuous integration in open-source projects. In *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Singapore, 426–437.
- [27] Yasutaka Kamei, Akito Monden, Shinsuke Matsumoto, Takeshi Kakimoto, and Ken-ichi Matsumoto. 2007. The effects of over and under sampling on fault-prone module detection. In *First international symposium on empirical software engineering and measurement (ESEM 2007)*. IEEE, Madrid, Spain, 196–204.
- [28] Passant Kandil, Sherin Moussa, and Nagwa Badr. 2017. Cluster-based test cases prioritization and selection technique for agile regression testing. *Journal of Software: Evolution and Process* 29, 6 (2017), e1794.
- [29] Hae-Young Kim. 2014. Analysis of variance (ANOVA) comparing means of more than two groups. *Restorative dentistry & endodontics* 39, 1 (2014), 74–77.
- [30] Jung-Min Kim and Adam Porter. 2002. A history-based test prioritization technique for regression testing in resource constrained environments. In *Proceedings of the 24th international conference on software engineering*. IEEE, Orlando, FL, USA, 119–129.
- [31] Miroslav Kubat, Stan Matwin, et al. 1997. Addressing the curse of imbalanced training sets: one-sided selection. *Icml* 97, 1 (1997), 179.
- [32] Adriaan Labuschagne, Laura Inozemtseva, and Reid Holmes. 2017. Measuring the cost of regression testing in practice: a study of Java projects using continuous integration. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, United States, 821–830.
- [33] Jorma Laurikkala. 2001. Improving identification of difficult small classes by balancing class distribution. In *Conference on artificial intelligence in medicine in Europe*. Springer, Cascais, Portugal, 63–66.
- [34] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. 2017. Imbalanced-learn: a Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research* 18, 17 (2017), 1–5. <http://jmlr.org/papers/v18/16-365>
- [35] Jackson A Prado Lima and Silvia R Vergilio. 2020. Test case prioritization in continuous integration environments: a systematic mapping study. *Information and Software Technology* 121 (2020), 106–268.
- [36] Inderjeet Mani and I Zhang. 2003. kNN approach to unbalanced data distributions: a case study involving information extraction. In *Proceedings of workshop on learning from imbalanced datasets*, Vol. 126. ICML, Washington, DC, 1–7.
- [37] Kenneth O McGraw and Seok P Wong. 1992. A common language effect size statistic. *Psychological bulletin* 111, 2 (1992), 361.
- [38] Donald Metzler and W. Bruce Croft. 2007. Linear feature-based models for information retrieval. *Information Retrieval* 10, 3 (01 Jun 2007), 257–274. <https://doi.org/10.1007/s10791-006-9019-z>
- [39] Hien M Nguyen, Eric W Cooper, and Katsuari Kamei. 2009. Borderline over-sampling for imbalanced data classification. In *Proceedings: Fifth International Workshop on Computational Intelligence & Applications*. IEEE SMC Hiroshima Chapter, Hiroshima, Japan, 24–29.
- [40] Cong Pan and Michael Pradel. 2021. Continuous test suite failure prediction. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, Denmark, 553–565.
- [41] Rongqi Pan, Mojtaba Bagherzadeh, Taher Ahmed Ghaleb, and Lionel Briand. 2021. Test case selection and prioritization using machine learning: a systematic literature review. *Empirical Software Engineering* 27, 29 (2021), 1–34.

- [42] Qianyang Peng, August Shi, and Lingming Zhang. 2020. Empirically revisiting and enhancing IR-based test-case prioritization. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual Event, USA) (ISSTA 2020). Association for Computing Machinery, New York, NY, USA, 324–336.
- [43] JC Riquelme, R Ruiz, D Rodríguez, and J Moreno. 2008. Finding defective modules from highly unbalanced datasets. *Actas de los Talleres de las Jornadas de Ingeniería del Software y Bases de Datos 2*, 1 (2008), 67–74.
- [44] Gregg Rothermel, Roland H Untch, Chengyun Chu, and Mary Jean Harrold. 1999. Test case prioritization: an empirical study. In *Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99)*. Software Maintenance for Business Change (Cat. No. 99CB36360). IEEE, Oxford, UK, 179–188.
- [45] Rui Shu, Tianpei Xia, Laurie Williams, and Tim Menzies. 2022. Dazzle: using optimized generative adversarial networks to address security data class imbalance issue. In *MSR*. ACM, Pittsburgh, PA, USA, 1–12.
- [46] Michael R Smith, Tony Martinez, and Christophe Giraud-Carrier. 2014. An instance level analysis of data complexity. *Machine learning* 95, 2 (2014), 225–256.
- [47] Helge Spieker, Arnaud Gotlieb, Dusica Marijan, and Morten Mossige. 2017. Reinforcement learning for automatic test case prioritization and selection in continuous integration. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, United States, 12–22.
- [48] Chakkrit Tantithamthavorn, Ahmed E Hassan, and Kenichi Matsumoto. 2018. The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. *IEEE Transactions on Software Engineering* 46, 11 (2018), 1200–1219.
- [49] Ivan Tomek. 1976. An experiment with the edited nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 6, 6 (1976), 448–452.
- [50] Ivan Tomek. 1976. Two modifications of CNN. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* SMC-6, 11 (1976), 769–772.
- [51] Bernard L Welch. 1947. The generalization of student's' problem when several different population variances are involved. *Biometrika* 34, 1/2 (1947), 28–35.
- [52] Dennis L Wilson. 1972. Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics* SMC-2, 3 (1972), 408–421.
- [53] Qiang Wu, Christopher J. C. Burges, Krysta M. Svore, and Jianfeng Gao. 2010. Adapting boosting for information retrieval measures. *Information Retrieval* 13, 3 (01 Jun 2010), 254–270.
- [54] Ahmadreza Saboor Yaraghi, Mojtaba Bagherzadeh, Nafiseh Kahani, and Lionel Briand. 2022. Scalable and Accurate Test Case Prioritization in Continuous Integration Contexts. *IEEE Transactions on Software Engineering* 20 (2022), 1–27. <https://doi.org/10.1109/TSE.2022.3184842>
- [55] Yan-Ping Zhang, Li-Na Zhang, and Yong-Cheng Wang. 2010. Cluster-based majority under-sampling approaches for class imbalance learning. In *2010 2nd IEEE International Conference on Information and Financial Engineering*. IEEE, Chongqing, China, 400–404.